

Proposta Técnica: Correção de Direção e Controle de Motores (UD-H1)

Este documento detalha o diagnóstico técnico e as alterações de código propostas para resolver o problema do robô **UD-H1** puxar para o lado, derrapar (*skidding*) ou oscilar ao andar, garantindo que ele siga as direções comandadas de forma precisa e linear.

1. Diagnóstico do Problema (O que estava acontecendo?)

A equipe identificou que o robô apresentava dificuldades em manter uma trajetória retilínea estável. A nossa análise do firmware do Arduino e do driver do ROS 2 revelou **quatro causas principais** para essa instabilidade:

A. Frequência de Controle Muito Baixa (10 Hz)

A malha de controle no Arduino rodava a cada **100ms**. Em motores de hoverboard (que possuem torque e velocidade elevados), essa latência de 0,1s é excessiva. O robô desviava fisicamente antes que o Arduino pudesse ler o erro e ajustar a potência, provocando guinadas e derrapagens.

B. Controle Apenas Integral (I-only)

A fórmula do controle de velocidade era calculada acumulando o erro direto no PWM (`__CODEPLACEHOLDER0__`). Matematicamente, isso se comporta apenas como um termo **Integral**. Controladores puramente integrais têm resposta lenta para iniciar o movimento (rampa lenta) e sofrem de *overshoot* (passam do valor alvo e ficam oscilando em "S").

C. Conflito entre Malha Fechada e TRIM Estático

Havia uma redução estática de potência configurada no motor direito (`__CODEPLACEHOLDER0__`). Como o robô está em malha fechada, se o TRIM força a roda a girar mais devagar, o controlador lê a velocidade menor e aumenta o PWM para compensar. Isso gerava uma disputa constante entre a atenuação do TRIM e a correção da malha fechada, causando instabilidade.

D. Negação Global de Sinais no ROS 2

Havia uma negação de eixos na odometria (`__CODEPLACEHOLDER0__`). Isso fazia com que, ao comandar o robô para frente no teclado, ele andasse fisicamente para frente, mas o ROS reportasse que ele estava indo para trás no RViz. Essa contradição de sinais quebrava a consistência matemática do AMCL e do Nav2, forçando o robô a tentar corrigir curvas inexistentes.

2. Alterações Propostas e Implementadas

As seguintes melhorias foram aplicadas no repositório local:

A. Novo Firmware do Arduino

- **Controle a 50 Hz (20ms):** Reduzida a latência de controle em 5 vezes, permitindo correções rápidas de trajetória.
- **Controlador PI + Feedforward:** Implementada a fórmula completa de controle Proporcional-Integral mais um ganho de Feedforward (`__CODEPLACEHOLDER0__`). Isso faz com que os motores respondam instantaneamente ao comando de velocidade alvo (Feedforward), cabendo ao PI apenas a correção de pequenas perturbações do solo.
- **Interrupções de Hardware Nativas:** O código foi alterado para usar `__CODEPLACEHOLDER0` **direto de hardware nos pinos 2 e 3** (`CODEPLACEHOLDER1` e `CODEPLACEHOLDER2`) **em vez da biblioteca** `CODEPLACEHOLDER3`, o que elimina a possibilidade de perda de pulsos do encoder em velocidades altas.
- **Zerar Integradores:** Integradores de erro são reiniciados ao parar o robô, prevenindo acúmulo de potência residual.
- **Remoção do TRIM:** TRIM estático zerado, permitindo que a malha de controle controle a linearidade.

B. Novo Driver ROS 2

- **Rampas de Aceleração Suaves:** Implementamos limites dinâmicos de aceleração linear (0.8 m/s^2) e angular (1.5 rad/s^2). Mesmo que receba comandos bruscos do teclado ou planejador, o robô acelerará de forma suave, evitando derrapagens por perda de tração.
- **Remoção de Negações:** Odometria linear e angular agora reportam valores estritamente positivos ao andar para a frente e girar para a esquerda (anti-horário), em conformidade total com o padrão ROS.

C. Utilitário Customizado WASD

Criamos o nó `__CODEPLACEHOLDER0__` para substituir o teclado padrão do ROS. A equipe agora controla o robô usando a convenção de jogos padrão no terminal:

- `__CODEPLACEHOLDER0 / CODEPLACEHOLDER1__` : Ir para frente / trás (velocidade segura de 0.2 m/s)
- `__CODEPLACEHOLDER0 / CODEPLACEHOLDER1__` : Girar sobre o próprio eixo para a esquerda / direita (0.6 rad/s)
- `__CODEPLACEHOLDER0__` : Parar o robô instantaneamente.

3. Roteiro de Teste Proposto para a Equipe

Para validar as alterações fisicamente, a equipe deve seguir estes 3 passos simples no laboratório:

Passo 1: Teste com Rodas Suspensas (Estático)

1. Colocar o robô em cima de um suporte para que as rodas girem livremente no ar.
2. Compilar o pacote e rodar a teleoperação:

```
colcon build --packages-select serial_com_py
source install/setup.bash
ros2 run serial_com_py wasd_teleop
```

3. Pressionar **W** e verificar se as duas rodas giram na mesma velocidade para a frente. Pressionar **S** e verificar se giram para trás.
4. Pressionar **A** e verificar se a roda esquerda gira para trás e a direita gira para frente (sentido de rotação para a esquerda).

Passo 2: Teste de Consistência de Sentido no RViz2

1. Abrir a visualização do robô (__CODEPLACEHOLDER0__).
2. Empurrar o robô fisicamente para frente e verificar se a odometria e o laser se deslocam para frente na tela.
3. Rotacionar o robô fisicamente para a esquerda e verificar se a seta vermelha de odometria gira para a esquerda.

Passo 3: Teste de Pista de 1 Metro (Dinâmico)

1. Posicionar o robô no chão, alinhado com uma fita métrica.
2. Comandar o robô para avançar a __CODEPLACEHOLDER0__ (segurando a tecla **W**).
3. O robô deve andar exatamente 1 metro em linha reta. O desvio lateral medido em relação à linha guia deve ser **menor do que 5 cm**.